Klassenmodell: aktueller Nachtrag

Aktualisierung: 28.10.2026 · **Quellcodebasis:** `02eb79f8`

Stand: 28.10.2026 · **Basis:** C++17, optionale SFML-3.1.0-GUI Quellcodeabgleich: `include/` und `src/` am Commit `02eb79f8`.

Zwei ausführbare Programme

Programm	Einstieg	Steuerung	Darstellung	Zusätzliche Abhängigkeit
`vier-gewinnt`	`src/main.cpp`	`Game`	Funktionen im Namensraum `console_ui`	Keine SFML-Abhängigkeit
`vier-gewinnt-gui`	`src/gui/Main.cpp`	`App` und `MatchSession`	SFML-Fenster, Texte, Hanteln, Schaltflächen	SFML Graphics und Audio 3.1.0

Die Konsole verwendet nicht `MatchSession`. Sie behält ihren bestehenden Controller `Game`. Die GUI nutzt eine eigene, ohne SFML testbare `MatchSession`. Das gemeinsame `Board` stellt die Regeln bereit. `AtomicFile.h` enthält die gemeinsame Datei-Hilfsfunktion. Das verhindert eine falsche Aussage über eine bereits vollständig vereinheitlichte Spielflussarchitektur.

Die editierbare Architekturübersicht steht in docs/ARCHITECTURE.md. Die Beziehungen sind in der folgenden Tabelle erläutert.

Verantwortlichkeiten und Beziehungen

Baustein	Wichtige Daten	Wesentliche Methoden und Funktionen
`Board`	`rows_`, `columns_`, `grid_`	`nextFreeRow`, `drop`, `undo`, `hasWon`, `winningCells`, `isFull`
`Player`	Name, Symbol	`getName`, `getSymbol`
`Game`	Ein Board, 2-4 Player, `vector<Move>`, Highscore-Map, Rundenzustand	`run`, `newGame`, `play`, `rematch`, `saveGame`, `loadGame`
`MatchSession`	Ein Board, 2-4 Player, `vector<GymMove>`, Score-Map, Gewinnlinie	`begin`, `drop`, `undo`, `rematch`, `save`, `load`, `saveScores`
`Move` und `GymMove`	`row`, `column`, `player`	Öffentliche Datenstrukturen für die jeweilige Historie
`console_ui`	Darstellungsfunktionen und Ansichtsdaten	`arena`, `present`, `result`, `animateDrop`
`App`	Fenster, Schrift, Schullogo, `MatchSession`, optionaler `GymBeat`	Ereignisschleife, `act`, `move`, `render`, `toggleMusic`
`GymBeat`	SoundBuffer, danach erzeugter Sound	Töne erzeugen, `play`, `stop`, `playing`
`gym::writeSafely`	Temporärer Zielpfad	Vollständig schreiben, prüfen und vorhandene Datei ersetzen

`Game` und `MatchSession` besitzen jeweils ihr Board und ihre Spieler. Es gibt kein gemeinsames, gleichzeitig benutztes Board-Objekt zwischen zwei gestarteten Programmen. `Player` enthält keinen Siegstand. Siege stehen in `Game::highscores_` beziehungsweise `MatchSession::scores_`.

Ein gültiger Spielzug

Die Oberfläche wandelt eine Spaltenauswahl in einen nullbasierten Index um. `Board::nextFreeRow` sucht von unten nach Platz. `Board::drop` setzt ein Symbol. Der Controller merkt sich den Zug, prüft `hasWon`, danach `isFull` und wechselt bei weiterlaufender Partie den Spieler. Ungültige Züge ändern weder Raster noch Zugfolge.

In der GUI berechnet `App` zunächst die Zielzeile und zeigt bei aktiver Animation eine fallende Hantel. Erst nach dem Fall übernimmt `MatchSession::drop` den Zug in den Spielzustand. Währenddessen blockiert die GUI weitere Spielaktionen. Die Konsole erzeugt ihre Animationsbilder in `console_ui`, ohne eine Grafikbibliothek zu benötigen.

Gewinnerkennung

`Board::hasWon` zählt vom letzten Stein aus waagerecht, senkrecht sowie entlang beider Diagonalen. Jede Achse wird in beide Richtungen untersucht. Die Länge enthält den neuen Stein nur einmal. Ab vier zusammenhängenden Symbolen liegt ein Sieg vor. `winningCells` liefert die zu markierenden Koordinaten. Bei einer längeren zusammenhängenden Gewinnlinie können mehr als vier Felder markiert werden.

Speichern und Laden

Beide Programme verwenden `data/savegame.txt` im Format `GYM1` und `data/highscore.txt`. Ein Spielstand enthält Spielerzahl, aktuellen Spieler, Abmessungen, Namen, Symbole, Zugfolge und Raster. Beim Laden entsteht zuerst ein geprüfter Kandidat. Erst nach erfolgreichen Prüfungen ersetzt er den laufenden Zustand.

`gym::writeSafely` schreibt zuerst eine temporäre Datei im Zielordner. Unter Windows ersetzt `MoveFileExW` das Ziel, unter anderen unterstützten Systemen `std::filesystem::rename`. Tests prüfen auch blockierte Temp-Dateien. Gleichzeitige Prozesse werden nicht durch eine Sperrdatei koordiniert. Der Demonstrationsstarter führt GUI und Konsole deshalb nacheinander aus.

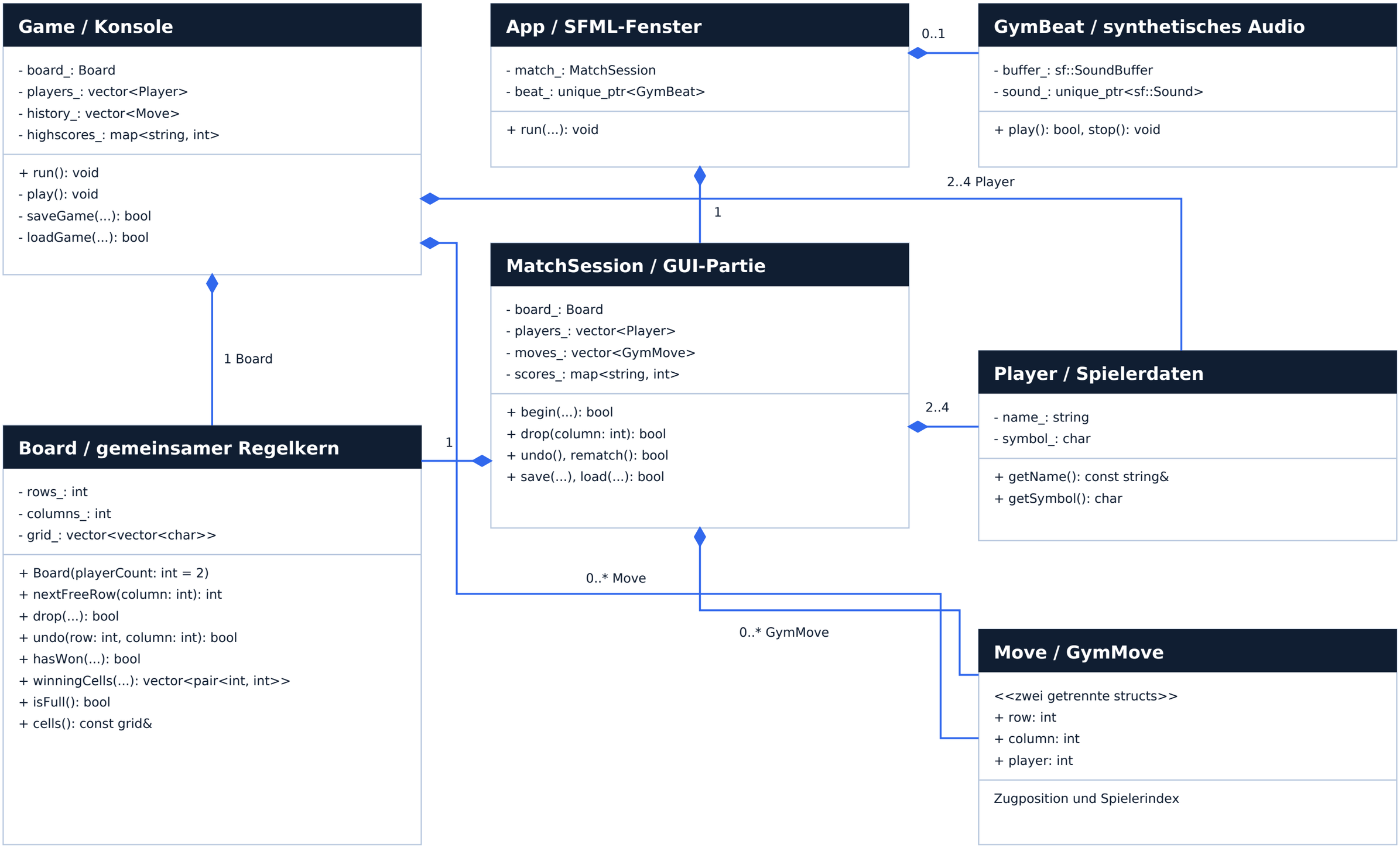
Grenzen und offene Arbeiten

- Das `GYM1`-Format speichert keine laufende Revanche-Rundennummer. Nach dem Laden beginnt die Anzeige bei Runde 1.
- Das 24-Byte-Limit für Namen ist bei UTF-8 nicht gleichbedeutend mit 24 sichtbaren Buchstaben.
- Board und Dateiersatz sind gemeinsam. Menüsteuerung und Teile der Persistenzvalidierung bestehen weiterhin in zwei Controllern.
- Es gibt keine 3D-Engine, keinen KI-Gegner und keinen Netzwerkmodus.
- Interaktive Zielgeräteprüfung und Unterrichtserklärung stehen noch aus.

Weitere Belege: Entwicklungsprotokoll, Qualität, GUI-Testplan.

Aktuelles Klassenmodell der Gym Edition

C++17 / Konsole und SFML-2D-GUI / ausgewählte Mitglieder / Quellcodebasis 02eb79f8 / 28.10.2026



Ausgefüllte Raute: Objektbesitz (Komposition). + public / - private. Game und MatchSession halten eigene Instanzen.
console_ui ist ein Namensraum mit freien Funktionen. gym::writeSafely ist eine Template-Funktion in AtomicFile.h und keine Klasse.
Julian Krauß / JK AXIS / mit KI-Unterstützung dokumentiert / vollständige Schnittstellen siehe include/ und src/gui/Main.cpp

Historischer Anhang

Aktualisierung: 28.10.2026 · **Quellcodebasis:** `02eb79f8`

Die folgenden Originalseiten dokumentieren den damaligen Planungs- und Lernstand vom 22.-25.10.2026. Sie bleiben als Entwicklungsgeschichte erhalten. Ihre Aussagen über fehlende GUI, offene GitHub-Veröffentlichung, ältere Menüs, Befehle und frühere Teststände sind keine aktuellen Anweisungen.

Für den Stand vom 28.10.2026 gelten der vorangestellte Nachtrag, die aktualisierte README und die Dateien unter docs/ im Entwicklungsbranch quality/gameplay-persistence-2026-10-28. Die alten 3D-Bilder sind Designstudien. Die tatsächlich ergänzte Oberfläche ist zweidimensional.

Gedruckte Seitenzahlen und Design-IDs im Anhang gehören zur ursprünglichen Fassung. Vorhandene Sprungziele werden beim Zusammenführen soweit vom PDF-Format unterstützt übernommen. Der neue PDF-Seitenbaum bietet zusätzlich einen direkten Einstieg in den Archivteil.

BERND BLINDOW GRUPPE
SCHULE AUSBILDUNG STUDIUM

PROGRAMMIERTECHNIK / 2026-2027

UML-Klassendiagramm

VIER GEWINNT · GYM EDITION

Objektorientierte Architektur des C++17-Lernprojekts

IST-MODELL
Game · Board · Player · Move

PROJEKTSTATUS
C++-Lernprojekt · kein GUI-Release

A3-DIAGRAMM · SEITE 2 | QUELLCODE-ABGLEICH | UML / OOP-LEGENDE

Das enthält diese Ausarbeitung

- **01 Gesamtübersicht**
  Alle vier Typen mit Sichtbarkeit, Methoden und drei Kompositionsbeziehungen.
- **02 Verantwortlichkeiten**
  Warum Game steuert, Board Regeln prüft und Player sowie Move Daten halten.
- **03 Beispielablauf**
  So arbeiten die Klassen bei einem gültigen Spielzug zusammen.
- **04 Abgleich und Abnahme**
  Welche Teile bereits im Code stehen und welche Erweiterungen geplant sind.

ZUM KLASSENDIAGRAMM SEITE 02 →

© 2026 Julian Krauß | JK-PT-2026-051-UML

DESIGN-ID JK-CD-DE-2026-003 · REV 3.2

VIER GEWINNT / GYM EDITION

UML-KLASSENDIAGRAMM

Ist-Architektur des C++17-Lernprojekts · Stand: 22. Oktober 2026

BERND BLINDOW GRUPPE
SCHULE AUSBILDUNG STUDIUM

4 TYPEN · 3 BEZIEHUNGEN | C++17 / KONSOLENAPP | WERK-ID JK-PT-2026-051-UML

Game

STEUERUNG

ATTRIBUTE

```
- chosenPlayers_: int = 2
- chosenNames_: vector<string>
- players_: vector<Player>
- board_: Board
- history_: vector<Move>
- current_: int = 0
- active_: bool = false
- animation_: bool = true
- highscores_: map<string,int>
```

METHODEN

```
+ Game()
+ run(): void
- configurePlayers(): void
- configureNames(): void
- newGame(): void
- play(): void
- showHelp(): void
- showInfo(): void
- showHighscores(): void
- saveGame(path: string): bool
- loadGame(path: string): bool
- loadHighscores(): void
- saveHighscores(): bool
- addWin(name: string): void
```

1 — board_

Board

SPIELFELD

ATTRIBUTE

```
- rows_: int
- columns_: int
- grid_: vector<vector<char>>
```

METHODEN

```
- inside(row:int,col:int): bool
- countDirection(...): int
+ Board(playerCount: int = 2)
+ rows(): int
+ columns(): int
+ tokenCount(): int
+ nextFreeRow(column:int): int
+ drop(col:int,sym:char,row:int&): bool
+ undo(row:int,col:int): bool
+ hasWon(row:int,col:int,sym:char): bool
+ isFull(): bool
+ cells(): const grid&
+ print(previewCol:int=-1,...): void
+ animateDrop(col:int,sym:char,row:int): void
```

0..* history_ — 1

2..4 — players_

Player

SPIELERDATEN

ATTRIBUTE

```
- name_: string
- symbol_: char
```

METHODEN

```
+ Player(name:string,symbol:char)
+ getName(): const string&
+ getSymbol(): char
```

Move

«struct» ZUGDATEN

ATTRIBUTE

```
+ row: int
+ column: int
+ player: int
```

UML-Notation: + public - private Komposition / Teil gehört zur Game-Instanz

IST-STAND · keine grafische GUI-Klasse

© 2026 Julian Krauß · JK-CD-DE-2026-003 / JK-PT-2026-051-UML

A3 / skalierbare Vektorgrafik

02 / KLASSENVERANTWORTUNG

Vier Typen. Klare Aufgaben.

Die Klassennamen und Funktionen stammen aus den vorhandenen Header-Dateien des C++17-Lernprojekts.

Game / Orchestrierung

Verarbeitet Menübefehle, verwaltet die Spielerreihenfolge, hält die Partie aktiv und koordiniert Spielstände sowie Highscores.

Board / Spielregeln

Besitzt das zweidimensionale Raster; führt Fallbewegung, Vorschau, Rücknahme, Gewinnprüfung und Konsolenausgabe aus.

Player / Identität

Speichert Namen und Symbol eines Spielers. Die Siegzahl wird im aktuellen Stand nicht hier abgelegt.

Move / Historie

Ein einfacher struct mit Zeile, Spalte und Spielerindex. Game speichert Zug-Einträge in einem vector.

BEZIEHUNGEN / KOMPOSITION

Game besitzt genau ein Board-Objekt (1), je nach gewähltem Modus zwei bis vier Player-Objekte (2..4) und null bis beliebig viele Move-Einträge (0..*).

Weil die Objekte als Member bzw. als Elemente von Member-Vektoren zur Game-Instanz gehören, sind ausgefüllte Rauten (Komposition) ein passendes UML-Modell.

© 2026 Julian Krauß | JK-PT-2026-051-UML
DESIGN-ID JK-CD-DE-2026-003 · REV 3.2

03 / SICHTBARKEIT / TYPEN

So liest du das Diagramm

Die Notation zeigt Attribute, Methoden, Sichtbarkeit und Beziehungen. UML bleibt vom Quellcode unterscheidbar.

- private

Interne Daten und Hilfsmethoden. Beispiel: - grid_: vector<vector<char>> ist nur innerhalb von Board zugreifbar.

+ public

Öffentliche Methoden bilden die nutzbare Schnittstelle. Beispiel: + drop(...) gibt bool zurück.

Komposition

Das ausgefüllte Rautensymbol steht auf der Besitzer-Seite, hier bei Game. Multiplikitäten: 1, 2..4 und 0..*.

«struct» Move

Move besitzt drei öffentliche Datenfelder. Es ist ein eigener Datentyp, kein zusätzlicher Spielcontroller.

BEISPIEL / C++-DEKLARATION → UML

```
private: Board board_;
public: void run();
// Game.h: vorhandene Deklarationen
```

```
UML:  - board_: Board
UML:  + run(): void
```

Die UML-Schreibweise nennt zuerst den Methodennamen und nach dem Doppelpunkt den Rückgabetyp. Das Beispiel entspricht den vorhandenen Game-Deklarationen.

© 2026 Julian Krauß | JK-PT-2026-051-UML

04 / ZUSAMMENSPIEL

Vom Menü bis zum Sieg

Beispiel für einen regulären Spielzug. Jede Stufe lässt sich im Sourcecode unmittelbar wiederfinden.

01

main.cpp

`Game game; game.run();`

Die Instanz entsteht. run() startet die Hauptmenü-Schleife.

02

Game::play()

`Eingabe und Zug validieren`

Game erkennt die Spalte und bestimmt den aktiven Spieler.

03

Board::drop()

`Kurzhantel im Raster ablegen`

Board berechnet die freie Zeile und aktualisiert genau eine Zelle.

04

Board::hasWon()

`Viererreihe überprüfen`

Board zählt in vier Richtungsachsen vom neuen Stein aus.

05

Game::addWin()

`Ergebnis dauerhaft erfassen`

Nur bei einem Sieg wird der Highscore erhöht und gespeichert.

Ohne Gewinn prüft Game auf Remis und wechselt weiter. Eine ungültige Eingabe verändert das Spielfeld nicht.

© 2026 Julian Krauß | JK-PT-2026-051-UML
DESIGN-ID JK-CD-DE-2026-003 · REV 3.2

05 / NACHVOLLZIEHBARKEIT

Was ist im Code? Was ist geplant?

Für die Vorführung ist die Unterscheidung zwischen tatsächlichem C++-Code und optionalem Grafikdesign wichtig.

BAUSTEIN	QUELLCODE	ZUORDNUNG
Player	include/Player.h	Spielername, Symbol
Board	include/Board.h	Feld, Prüfungen, Animation
Game	include/Game.h	Menü, Spiel, Persistenz
Move	include/Game.h	Rückgängig-Historie
main()	src/main.cpp	Programmstart; keine Klasse

Wichtige Modellentscheidung

Die Schulaufgabe zeigt beispielhaft Player::wins. Im vorhandenen Lernprojekt steht die Siegzahl dagegen in Game::highscores_ als map<string,int>. Das Diagramm zeigt den tatsächlichen Code.

Gaming-UI = separates Zukunftsthema

Derzeit besitzt Board eine ASCII-Kurzhantelausgabe und Terminal-Fallanimation. Eine echte GUI, ein 3D-Renderer oder zusätzliche Fensterklassen sind noch nicht vorhanden.

ABNAHME / ERKLÄRUNG IM UNTERRICHT

Warum verändert Game nicht selbst das Raster? Wozu dient history_? Wann speichert addWin() den Highscore? Diese Fragen solltest du am Code erklären können.

© 2026 Julian Krauß | JK-PT-2026-051-UML